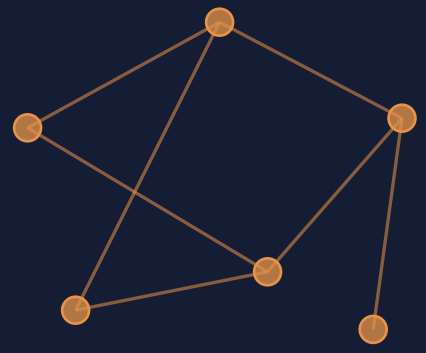
BAĞLI LİSTELER

3. Hafta — Dinamik Bellek, Tek/Çift Yönlü ve Dairesel Bağlı Listeler

Karadeniz Teknik Üniversitesi

Yazılım Geliştirme Bölümü

Dr. Öğr. Üyesi Sayyad Alizadeh



Geçen Haftadan Bağlantı

- Dizilerin sabit boyut ve pahalı ekleme/silme (O(n)) sınırlamalarını gördük.
- Öz-referanslı struct kavramını tanıtarak bu haftaya köprü kurmuştuk.
- **Bu hafta bu köprüyü tamamlıyoruz: struct Dugum artık tam bir veri yapısının temeli olacak.**

2. Haftadan Hatırlatma

```
struct Dugum {
    int veri;
    struct Dugum *sonraki;
};
```

Bu struct, artık “düğüm (node)” adını alacak ve dinamik olarak (malloc ile) oluşturulacak zincirleme bir yapının yapı taşı olacak.

Bu Haftanın Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler...

1. Bağlı listeyi diziyle karşılaştırıp ne zaman hangisinin uygun olduğuna karar verebilir
2. malloc/free ile dinamik bellek yönetimini doğru şekilde uygulayabilir
3. Tek yönlü bağlı listede ekleme/silme işlemlerini (baş/son/ara) adım adım uygulayabilir
4. Yanlış işlem sırasının bağlantı kaybına nasıl yol açtığını açıklayabilir
5. Çift yönlü ve dairesel bağlı listelerin temel farklarını tanıyabilir
6. Bellek sızıntısı ve sarkan işaretçi hatalarından nasıl kaçınılacağını bilir

Bu Haftanın Yol Haritası

TEK YÖNLÜ LİSTE

Kavram, oluşturma, gezinme, ekleme/silme (baş/son/ara)

Slayt 5–52

ÇİFT YÖNLÜ / DAİRESEL

prev/next, temel işlemler, dairesel gezinme

Slayt 53–57

BELLEK GÜVENLİĞİ + KAPANIŞ

Sızıntı, sarkan işaretçi, özet, quiz, ödev

Slayt 58–65

Bağlı Liste (Linked List) Nedir?

Her biri veri VE bir sonrakinin ADRESİNİ tutan, birbirine ZINCIRLEME bağlı dinamik düğümler (node) topluluğudur.



- Dizinin aksine, elemanlar bellekte BİTİŞİK OLMAK ZORUNDA DEĞİLDİR — adresler dizide gördüğünüz gibi rastgele sıralı değildir (1000, 3200, 1800...).
- **Bu esneklik dinamik büyüme/küçülme sağlar, ama doğrudan indeksli erişimi (dizi[i]) ortadan kaldırır.**

Dizi vs Bağlı Liste — Neden Önemli?

Dizi	Bağlı Liste
▪ Bitişik bellek	▪ Dağınık bellek (işaretçilerle bağlı)
▪ Sabit boyut	▪ Dinamik boyut
▪ Hızlı indeksli erişim: O(1)	▪ İndeksli erişim yok: O(n)
▪ Ortaya ekleme/silme: O(n)	▪ Başa ekleme/silme: O(1)

Altın Kural: Verinize çok sık İNDEKSLE erişiyorsanız dizi; verinizi sık sık BAŞA/ORTAYA ekliyor/siliyorsanız bağlı liste tercih edilir. Bu hafta bu ikinci senaryoyu derinlemesine işleyeceğiz.

Dinamik Bellek Yönetimi — malloc Kavramı

- Sabit boyutlu dizilerin kapasitesi oluşturulduktan sonra değişmez. Bağlı listelerde ise düğümler, çalışma zamanında ihtiyaç duyuldukça dinamik olarak oluşturulur ve serbest bırakılır.
- **Bu, C'nin standart kütüphanesindeki malloc() fonksiyonuyla yapılır.**

```
işaretçi = malloc(sizeof(tür));
```

malloc; istenen boyutta boş bir bellek bloğu bulur, onu REZERVE eder ve BAŞLANGIÇ ADRESİNİ (işaretçi olarak) döndürür. Yeterli bellek yoksa NULL döner — bu her zaman kontrol edilmelidir.

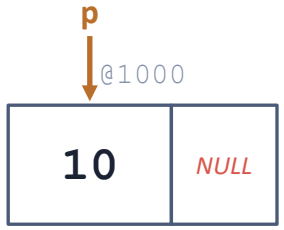
malloc — C Kodu ve Bellek Diyagramı

```c
struct Dugum *p;
p = malloc(sizeof(struct Dugum));

if (p == NULL) {
    printf("Bellek yetersiz!\n");
    exit(1);
}

p->veri = 10;
p->sonraki = NULL;
```

Oluşan Bellek Durumu



malloc, yığın (heap) adı verilen bellek bölgesinden bir blok ayırır ve adresini p'ye yazar. p artık bu yeni düğümü “gösteriyor”.

Yaygın Hata: malloc'un dönüş değerini (NULL olabilir) kontrol etmeden p->veri gibi bir erişim yapmak çökmeye yol açabilir.

free() ve Bellek Serbest Bırakma

- **malloc() ile ayrılan bellek, artık kullanılmayacaksa free() ile bellek yöneticisine geri verilmelidir. Bu işlem, belleğin doğrudan işletim sistemine iade edildiği anlamına gelmez.**

```
free(p);        // p'nin gösterdiği bellek serbest bırakılır
p = NULL;       // p artık geçersiz bir adresi göstermiyor
```

ÖNEMLİ: free(p) sadece BELLEĞİ serbest bırakır, p değişkeninin kendisini silmez. free()'den SONRA p, artık geçersiz bir adresi gösteren "sarkan işaretçi" (dangling pointer) haline gelir — bu yüzden hemen p=NULL yapılmalıdır (58-59. slaytlarda detaylı işlenecek).

Düğüm (Node) Yapısı

```c
struct Dugum {
    int veri;              // veri alanı
    struct Dugum *sonraki;  // bir sonraki düğümün adresi
};

typedef struct Dugum Dugum;   // kısaltma (opsiyonel)
```

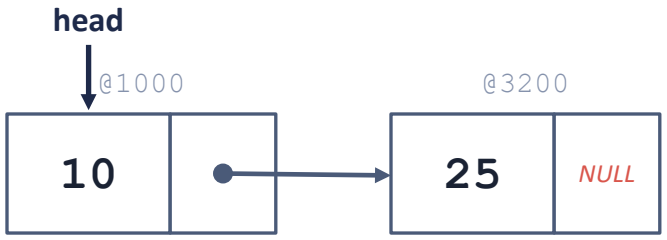


veri *sonraki*

Bu, 2. haftada gördüğümüz öz-referanslı struct'ın TAM olarak kendisi — farklı olan tek şey artık bu düğümlerin dinamik olarak (malloc ile) çoklu olarak oluşturulup zincirlenmesidir.

NULL Kavramı

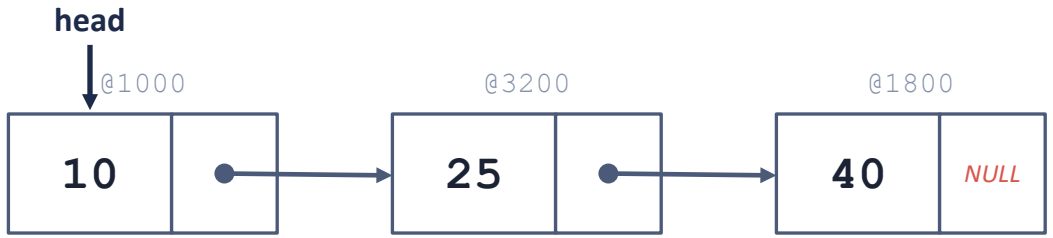
- NULL; “bu işaretçi hiçbir şeyi göstermiyor” anlamına gelen özel bir adres değeridir (genellikle 0).
- **Bir listenin SONUNU işaretlemek için kullanılır: son düğümün sonraki alanı NULL'dur.**



Yaygın Hata: NULL bir işaretçi üzerinden alan erişimi yapmak (ör. p->veri, p==NULL iken) programın çökmesine (segmentation fault) yol açar. Her gezinme işleminde NULL kontrolü şarttır.

head İşaretçisi Kavramı

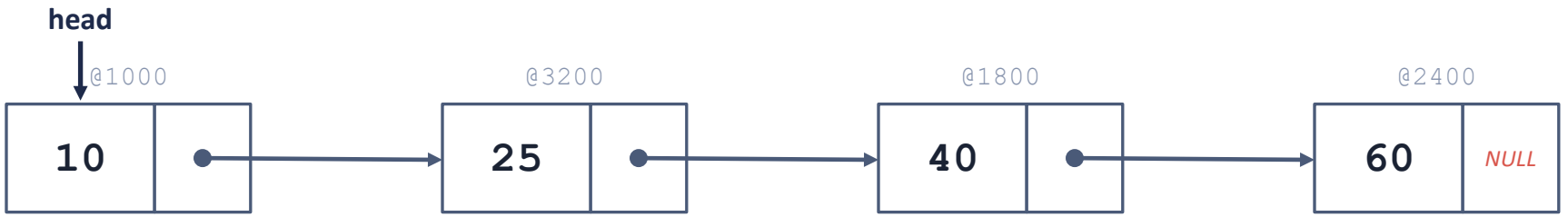
- head; listenin İLK düğümünün adresini tutan, listeye giriş noktası olan tek bir işaretçidir.
- **head'i kaybederseniz TÜM LİSTEYE erişimi kaybedersiniz — başka hiçbir yerden geri dönülemez!**



Boş bir liste, head'in NULL olduğu durumdur: struct Dugum *head = NULL;. Bu, listenin “hiç elemanı yok” durumunu temsil eder.

Tek Yönlü Bağlı Liste — Genel Görünüm

Bu hafta boyunca kullanacağımız örnek liste



Bu 4 düğümlü liste (10→25→40→60), bu hafta boyunca tüm işlemleri (ekleme, silme, arama) göstermek için KULLANILACAK ORTAK ÖRNEKTİR.

Adresler (1000, 3200, 1800, 2400) kasıtlı olarak sırasız seçilmiştir — gerçek dünyada malloc'un döndürdüğü adresler böyle öngörülemezdir.

Liste Oluşturma — Pseudocode

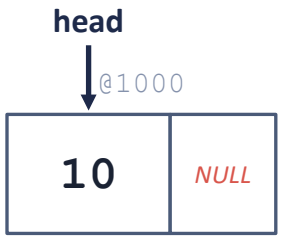
```
Fonksiyon YeniDugum(deger)
    yeni ← malloc(Dugum boyutu)
    eğer yeni == NULL
        dön HATA
    yeni.veri ← deger
    yeni.sonraki ← NULL
    dön yeni
```

Liste Oluşturma — C Kodu

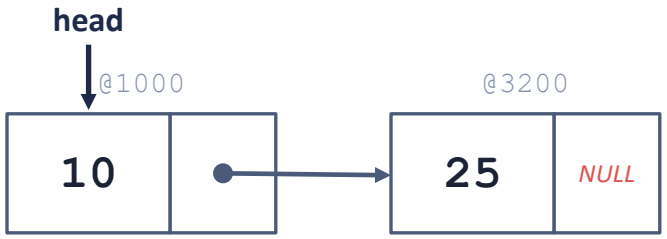
```
struct Dugum *yeniDugum(int deger) {
    struct Dugum *yeni = malloc(sizeof(struct Dugum));
    if (yeni == NULL) {
        return NULL;
    }
    yeni->veri = deger;
    yeni->sonraki = NULL;
    return yeni;
}
```

Liste Oluşturma — Görsel

struct Dugum *head = yeniDugum(10);



struct Dugum *ikinci = yeniDugum(25); head->sonraki = ikinci;



Liste böylece TEK TEK düğüm eklenerek büyür. Her yeni düğüm için bir önceki düğümün sonraki alanı güncellenmelidir — aksi halde bağlantı kurulmaz.

Gezinme (Traversal) — Pseudocode

```
Fonksiyon Gezin(head)
    p ← head
    iken p ≠ NULL
        yazdır p.veri
        p ← p.sonraki
```

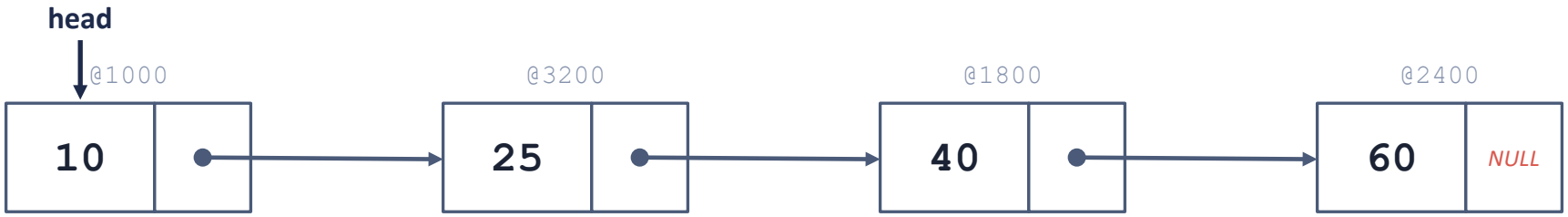
p, sırayla her düğümü “ziyaret eden” GEÇİCİ bir işaretçidir — head'in kendisi ASLA değiştirilmez, böyle head'i kaybetmeyiz.

Gezinme — C Kodu

```c
void gezin(struct Dugum *head) {
    struct Dugum *p = head;
    while (p != NULL) {
        printf("%d -> ", p->veri);
        p = p->sonraki;
    }
    printf("NULL\n");
}
```

Çıktı: 10 -> 25 -> 40 -> 60 -> NULL

Gezinme — Görsel Adım Adım



p'nin döngü boyunca ilerleyişi:

① p ② p ③ p ④ p

Adım 1: p=head(10). Adım 2: p=p->sonraki(25). Adım 3: p=p->sonraki(40). Adım 4: p=p->sonraki(60). Adım 5: p=p->sonraki=NULL → döngü biter.

Ekleme Türlerine Genel Bakış

Başa Ekleme

Yeni düğüm, listenin en başına eklenir. head değişir.

Sona Ekleme

Yeni düğüm, listenin en sonuna eklenir. Sona kadar gezinmek gerekir.

Araya Ekleme

Yeni düğüm, iki düğüm arasına eklenir. En hassas işlemdir.

Üçünde de ORTAK mantık: önce yeni düğüm malloc ile oluşturulur, sonra işaretçiler DOĞRU SIRAYLA güncellenir. Sıra yanlış olursa liste parçalanır — bunu 30-37. slaytlarda somut olarak göreceğiz.

Başa Ekleme — Pseudocode

```
Fonksiyon BaşaEkle(head, deger)
    yeni ← YeniDugum(deger)
    yeni.sonraki ← head
    head ← yeni
    dön head
```

head'in ilk kez elle değiştirildiği işlemdir. Yeni düğüm, ESKİ head'i göstermeli, SONRA head değişkeni yeni düğümü göstermelidir — sıra bu şekilde olmalı.

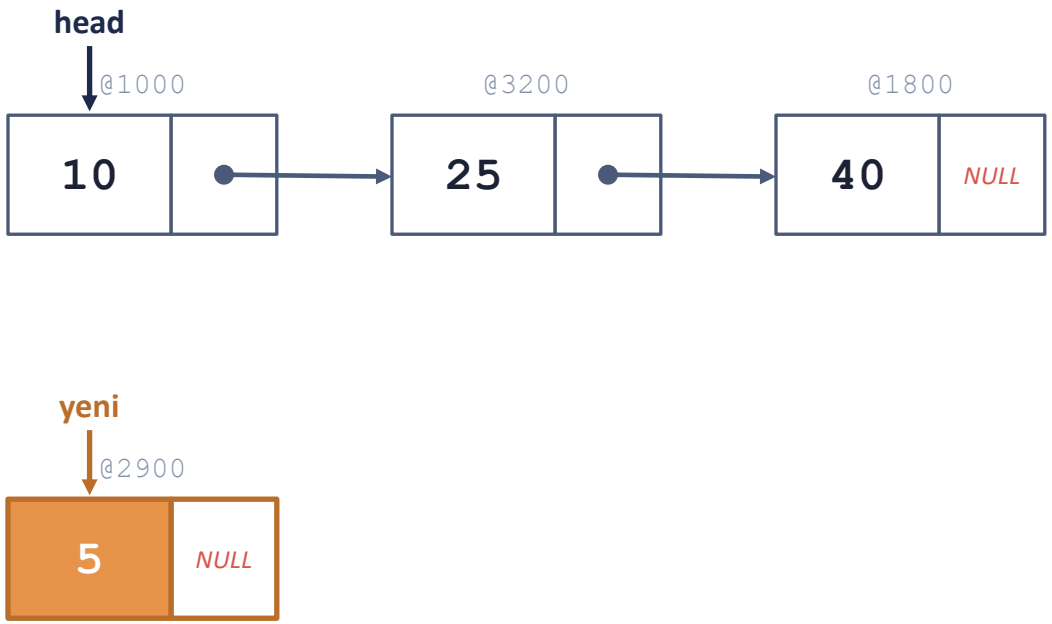
Başa Ekleme — C Kodu

```
struct Dugum *basaEkle(struct Dugum *head, int deger) {
    struct Dugum *yeni = yeniDugum(deger);
    if (yeni == NULL) {
        return head;   // bellek yetersiz, değişiklik yok
    }
    yeni->sonraki = head;
    head = yeni;
    return head;
}
```

Dikkat: C'de fonksiyon head'i DEĞER olarak alır. Güncellenmiş head'i kullanmak için çağırırken mutlaka: head = basaEkle(head, 5); yazılmalıdır.

Başa Ekleme — Adım 1 / 2

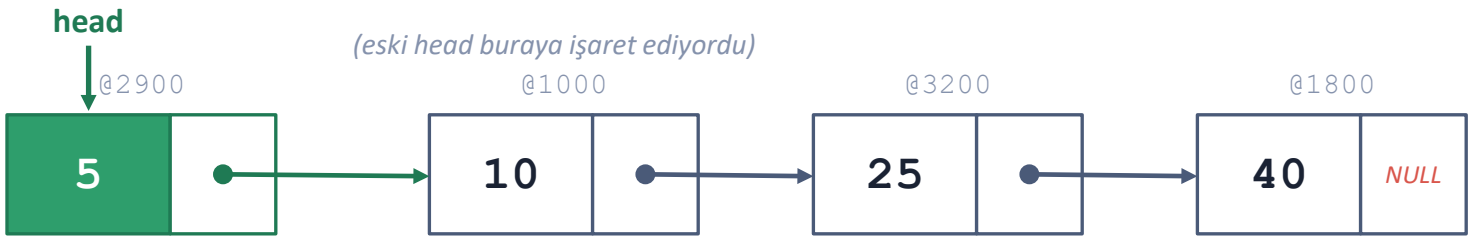
Yeni Düğüm Oluşturuldu (henüz bağlı değil)



yeniDugum(5) çağrıldı. Yeni düğüm bellekte var ama HENÜZ listeye bağlı değil — kendi başına duruyor.

Başa Ekleme — Adım 2 / 2

yeni->sonraki = head; head = yeni; — Son Durum



✓ Yeni düğüm (5) artık listenin başı. head değişkeni artık onu gösteriyor, o da eski ilk düğüme (10) bağlı. Liste: 5→10→25→40.

Başa Ekleme — Karmaşıklık Analizi

```
T(n) = 3 (sabit sayıda işlem)  →  O(1)
```

Girdi boyutundan (n) BAĞIMSIZDIR — liste ister 10 ister 10 milyon eleman içersin, başa ekleme her zaman AYNI 3 adımda tamamlanır. Bu, dizideki O(n) başa eklemeye göre önemli bir avantajdır (2. hafta).

Sona Ekleme — Pseudocode

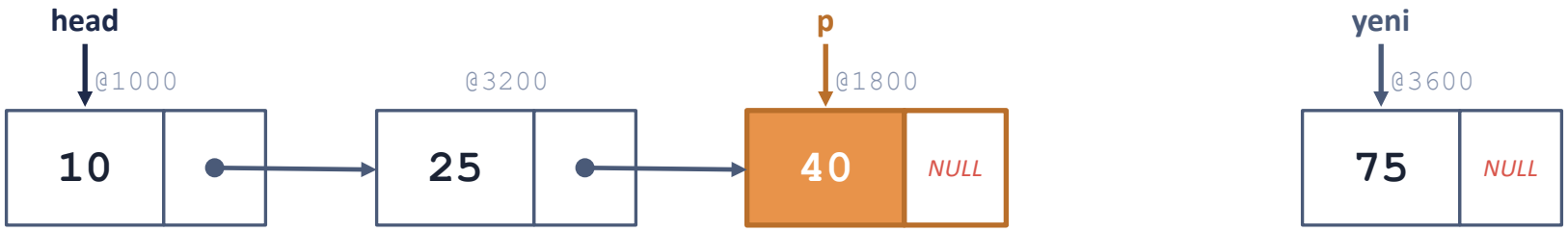
```
Fonksiyon SonaEkle(head, deger)
    yeni ← YeniDugum(deger)
    eğer head == NULL
        dön yeni   // liste boştu, yeni düğüm artık head
    p ← head
    iken p.sonraki ≠ NULL
        p ← p.sonraki
    p.sonraki ← yeni
    dön head
```

Sona Ekleme — C Kodu

```
struct Dugum *sonaEkle(struct Dugum *head, int deger) {
    struct Dugum *yeni = yeniDugum(deger);
    if (head == NULL) {
        return yeni;          // liste boştu
    }
    struct Dugum *p = head;
    while (p->sonraki != NULL) {
        p = p->sonraki;       // sona kadar git
    }
    p->sonraki = yeni;
    return head;
}
```

Sona Ekleme — Adım 1 / 2

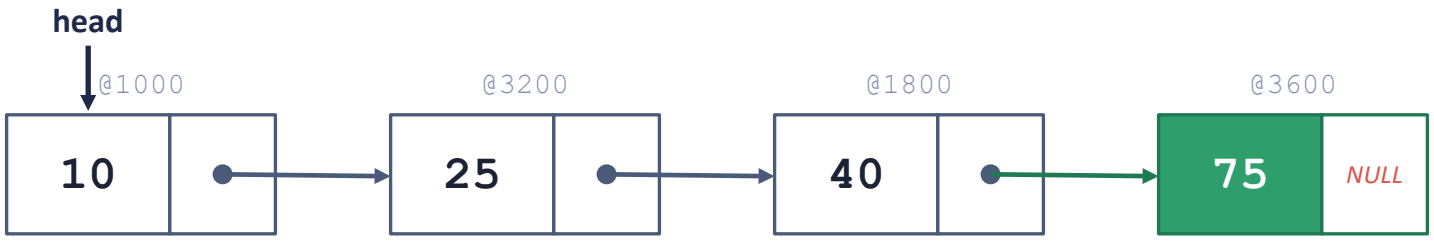
p, son düğüme kadar gezinir



p, head'ten başlayıp p->sonraki NULL olana kadar ilerler. Bu, p'nin SON düğüme (40) ulaştığı anı gösterir. Yeni düğüm (75) ayrıca oluşturulmuştur, henüz bağlı değil.

Sona Ekleme — Adım 2 / 2

p->sonraki = yeni; — Son Durum



✓ Artık son düğümün (40) sonraki alanı yeni düğümü (75) gösteriyor, ve 75'in sonraki alanı NULL. Liste: 10→25→40→75.

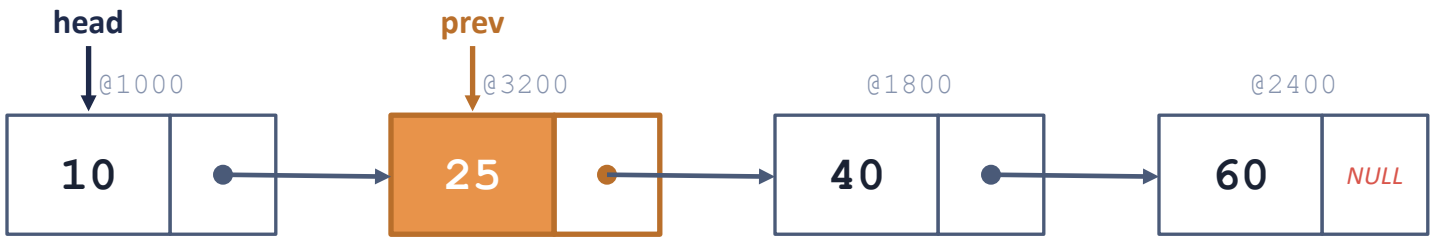
Sona Ekleme — Karmaşıklık Analizi

$$T(n) = n + c \quad \rightarrow \quad O(n)$$

Son düğüme ulaşmak için TÜM listeyi gezmek gerekir — tıpkı dizide gezinme gibi. İstisna: eğer listenin bir de TAIL (son düğüm) işaretçisi ayrıca tutuluyorsa, sona ekleme de O(1)'e düşer.

Araya Ekleme (İki Düğüm Arasına)

Problem Tanımı



- Hedef: değer=33'ü, 25 (prev) ile 40 arasına eklemek.
- **Ön Koşul: prev (kendinden SONRA ekleme yapılacak düğüm) zaten BİLİNMEKTEDİR — bu, karmaşıklık analizinde kritik bir varsayımdır.**
- Bu, bağlı listenin EN HASSAS işlemidir — doğru sıra hayati önem taşır.

Araya Ekleme — Pseudocode

```
Fonksiyon AraEkle(prev, deger)
    eğer prev == NULL
        dön HATA
    yeni ← YeniDugum(deger)
    yeni.sonraki ← prev.sonraki
    prev.sonraki ← yeni
```

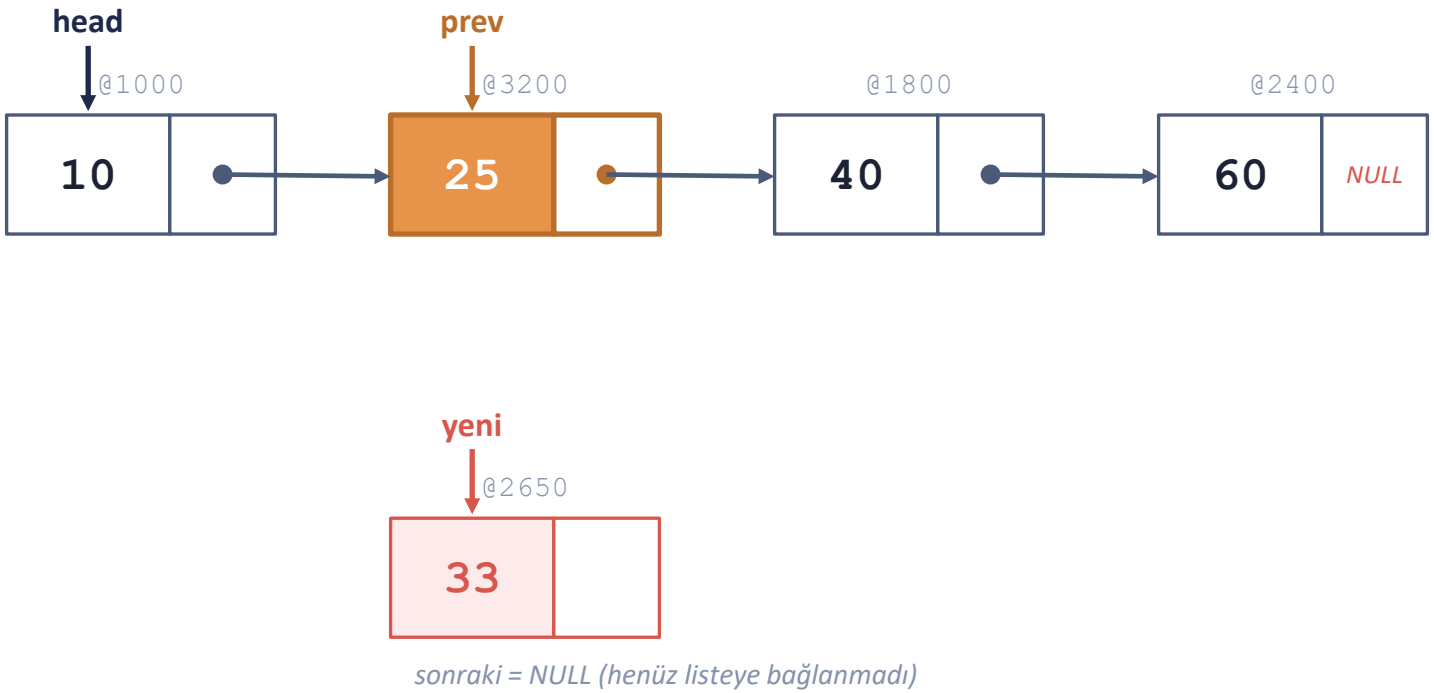
SIRA ÖNEMLİ: önce yeni.sonraki AYARLANIR, SONRA prev.sonraki DEĞİŞTİRİLİR. Bu iki satırın yeri değiştirilemez — nedenini birazdan göreceğiz.

Araya Ekleme — C Kodu

```
void araEkle(struct Dugum *prev, int deger) {
    if (prev == NULL) {
        return;              // geçersiz önceki düğüm
    }
    struct Dugum *yeni = yeniDugum(deger);
    if (yeni == NULL) {
        return;              // bellek yetersiz
    }
    yeni->sonraki = prev->sonraki;   // 1. önce bunu yap
    prev->sonraki = yeni;            // 2. sonra bunu yap
}
```

Araya Ekleme — Adım 1 / 3

Yeni Düğüm Oluşturuldu (henüz bağlı değil)

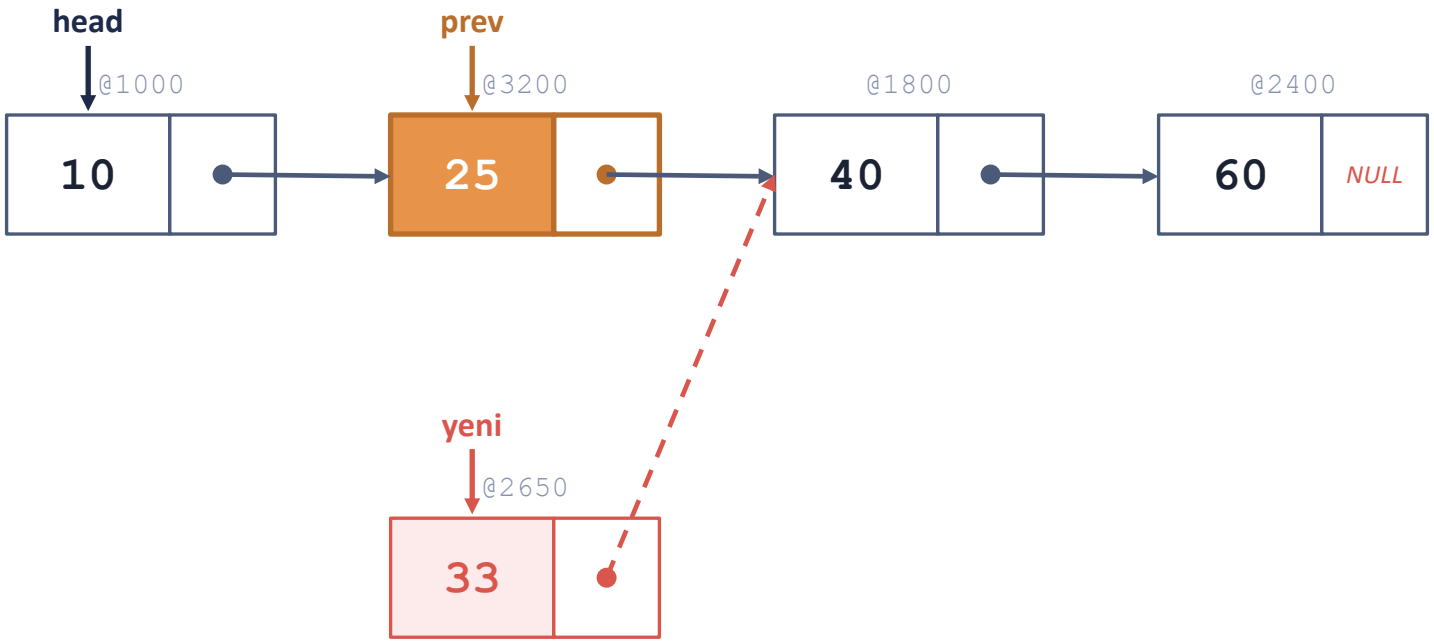


sonraki = NULL (henüz listeye bağlanmadı)

yeniDugum(33) çağrıldı. Liste HENÜZ değişmedi — 10→25→40→60 hala olduğu gibi duruyor. Yeni düğüm bağlı değil.

Araya Ekleme — Adım 2 / 3

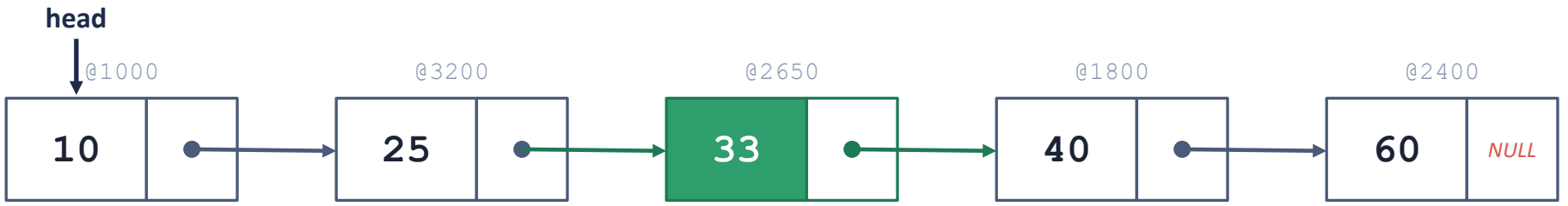
yeni->sonraki = prev->sonraki; (ara durum)



yeni->sonraki artık 40'ı gösteriyor (kesikli turuncu ok). ANCAK prev (25) HALA eskisi gibi 40'ı gösteriyor — liste HALA sağlam ve bozulmamış durumda!

Araya Ekleme — Adım 3 / 3

prev->sonraki = yeni; — Son Durum (DOĞRU Sıra)



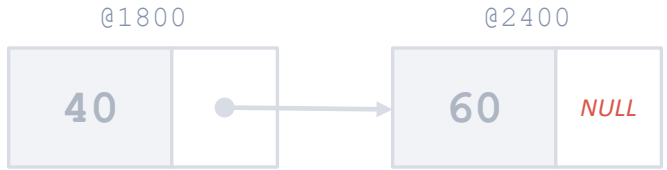
✓ prev (25) artık YENİ düğümü (33) gösteriyor — eski 40 bağlantısının üzerine yazıldı. Ama 33 zaten 40'ı gösterdiği için HİÇBİR BAĞLANTI KAYBOLMADI. Liste: 10→25→33→40→60.

HATA DURUMU: Sıra Ters Olursa

Önce prev->sonraki = yeni; yapılırsa (yeni->sonraki HENÜZ ayarlanmadan)



⚠ ARTIK ERİŞİLEMİYOR (bellek sızıntısı!)



Önce prev->sonraki = yeni yapılırsa eski bağlantının üzerine yazılır. Ardından yeni->sonraki = prev->sonraki çalıştırıldığında yeni düğüm kendisini gösterir. Eski sonraki düğümün adresi başka bir işaretçide saklanmamışsa listenin geri kalanına erişim kaybolur ve bellek sızıntısı oluşabilir.

Araya Ekleme — Karmaşıklık Analizi

prev ZATEN BİLİNİYORSA

Sadece 2 işaretçi ataması yapılır — girdi boyutundan bağımsız.

`O(1)`

prev'i BULMAK İÇİN ARAMA GEREKİYORSA

Önce prev'e kadar gezinmek gerekir (1. haftadaki doğrusal arama gibi).

`O(n)`

ÖNEMLİ AYRIM: Araya eklemenin “O(1)” olması SADECE prev düğümünü ZATEN elinizde tutuyorsanız geçerlidir. Değeri ile arama yapıp sonra eklemek isterseniz, arama kısmı O(n) olduğu için toplam işlem yine O(n) olur.

Baştan Silme — Pseudocode

```
Fonksiyon BaştanSil(head)
    eğer head == NULL
        dön NULL   // liste zaten boş
    gecici ← head
    head ← head.sonraki
    free(gecici)
    dön head
```

KRİTİK SIRA: silinecek düğümün adresi önce “gecici” değişkeninde SAKLANMALI, sonra head ilerletilmeli, EN SON free() çağrılmalı. Aksi halde free edilecek adresi kaybederiz!

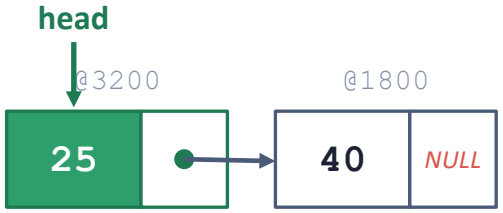
Baştan Silme — C Kodu ve Görsel

```
struct Dugum *bastanSil(
    struct Dugum *head) {
    if (head == NULL) {
        return NULL;
    }
    struct Dugum *gecici = head;
    head = head->sonraki;
    free(gecici);
    return head;
}
```

Önce (silinecek: 10)



Sonra (10 free edildi)



Liste: 10 → 25 → 40 listesinden ilk düğüm (10) silindi. Yeni liste: 25 → 40. Silinen düğümün belleği free() ile bellek yöneticisine geri verildi.

Sondan Silme — Pseudocode ve C Kodu

Pseudocode

```
Fonksiyon SondanSil(head)
    eğer head == NULL
        dön NULL
    eğer head.sonraki == NULL
        free(head)
        dön NULL    // tek düğümlüydü
    prev ← head
    iken prev.sonraki.sonraki ≠ NULL
        prev ← prev.sonraki
    free(prev.sonraki)
    prev.sonraki ← NULL
    dön head
```

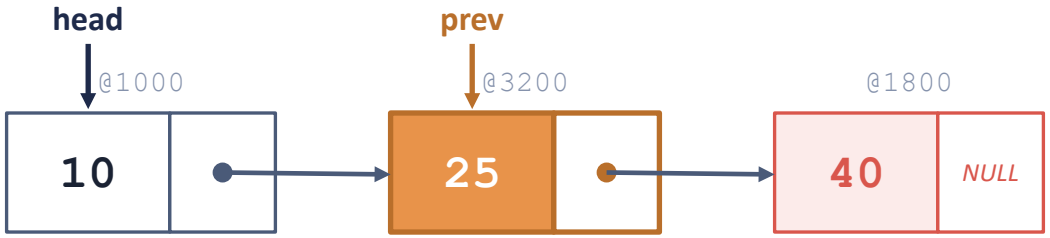
C Kodu

```
struct Dugum *sondanSil(struct Dugum *head) {
  if (head == NULL) return NULL;
  if (head->sonraki == NULL) {
    free(head);
    return NULL;
  }
  struct Dugum *prev = head;
  while (prev->sonraki->sonraki != NULL) {
    prev = prev->sonraki;
  }
  free(prev->sonraki);
  prev->sonraki = NULL;
  return head;
}
```

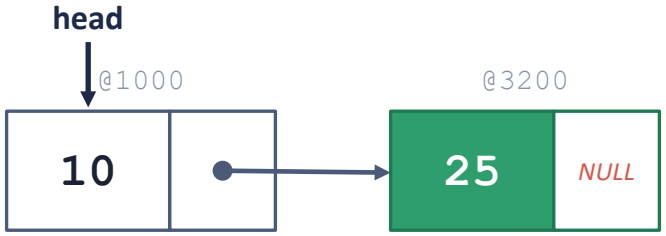
Zorluk: SONUNCUDAN BİR ÖNCEKİ düğüme ulaşmak gerekir — tek yönlü listede GERİYE bakma imkanı yoktur, bu yüzden prev'i takip etmek şarttır.

Sondan Silme — Görsel

Önce: prev, sonuncudan bir önceki düğüme kadar ilerler



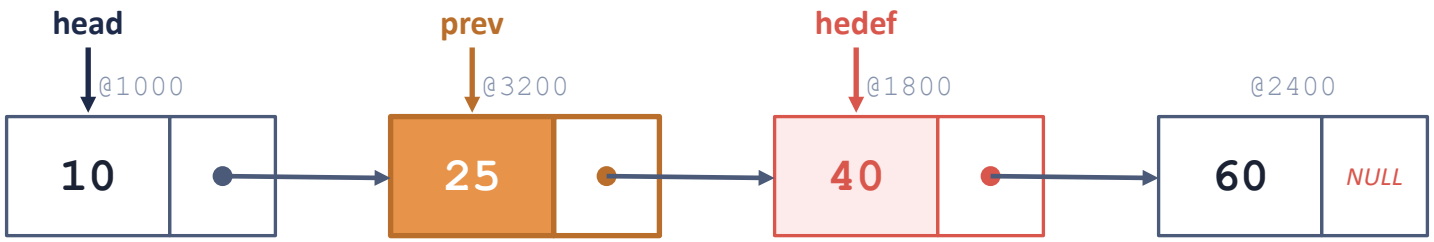
Sonra: free(prev->sonraki); prev->sonraki = NULL;



✓ Liste: 10→25→40 → 10→25. 40 free edildi, prev (25) artık yeni son düğüm ve next'i NULL.

Aradan Silme (İki Düğüm Arasından)

Problem Tanımı



- Hedef: değer=40 olan düğümü listeden çıkarıp belleğini serbest bırakmak.
- **Ön Koşul: prev (silinecek düğümün HEMEN ÖNCESİNDEKİ düğüm) bilinmektedir.**
- prev, hedef'i atlayarak doğrudan hedef'in sonrasına (60) bağlanmalı — SONRA hedef free edilmeli.

Aradan Silme — Pseudocode

```
Fonksiyon AradanSil(prev)
    eğer prev == NULL veya prev.sonraki == NULL
        dön HATA
    hedef ← prev.sonraki
    prev.sonraki ← hedef.sonraki
    free(hedef)
```

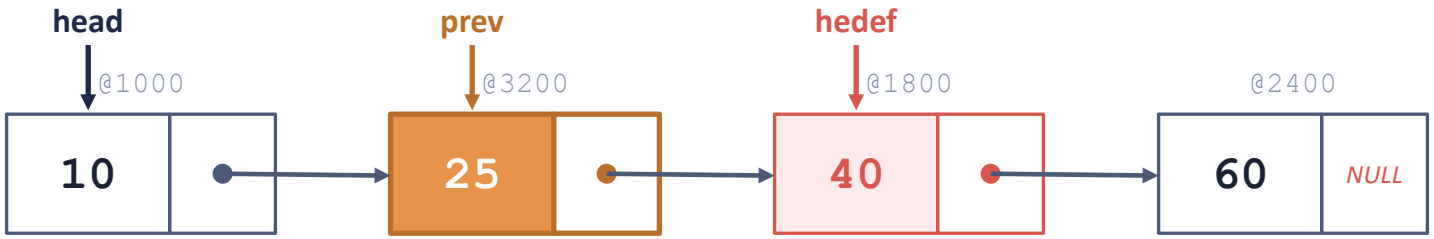
SIRA ÖNEMLİ: önce prev.sonraki güncellenir (hedef atlanır), SONRA free(hedef) çağrılır. Free önce yapılırsa hedef.sonraki değerine artık güvenle erişilemez!

Aradan Silme — C Kodu

```
void aradanSil(struct Dugum *prev) {
    if (prev == NULL || prev->sonraki == NULL) {
        return;                     // geçersiz durum
    }
    struct Dugum *hedef = prev->sonraki;
    prev->sonraki = hedef->sonraki;   // 1. önce atla
    free(hedef);                      // 2. sonra free
}
```

Aradan Silme — Adım 1 / 3

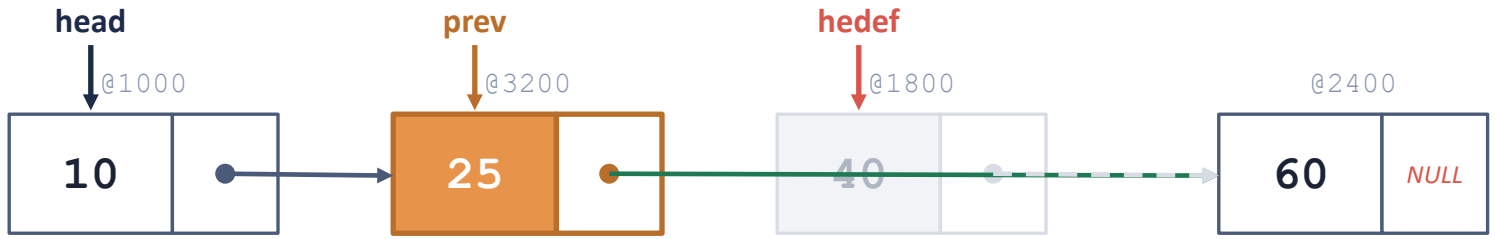
Başlangıç Durumu — prev ve hedef belirlendi



hedef = prev->sonraki; ile 40'ın adresi (1800) güvenli bir şekilde “hedef” değişkeninde saklanıyor. Liste henüz değişmedi: 10→25→40→60.

Aradan Silme — Adım 2 / 3

prev->sonraki = hedef->sonraki; (hedef hala bellekte, ama artık “atlanmış”)



prev (25) artık hedef'i (40) ATLAYIP doğrudan 60'ı gösteriyor (yeşil ok). hedef (40) hala bellekte duruyor ve kendi eski bağlantısını (kesikli gri ok) hala taşıyor, ama artık listenin BİR PARÇASI DEĞİL.

Aradan Silme — Adım 3 / 3

free(hedef); — Son Durum



✓ hedef (40) artık free edildi — belleği işletim sistemine iade edildi. Liste: 10→25→40→60 → 10→25→60. Bellek sızıntısı YOK, çünkü silmeden önce bağlantı doğru şekilde atlandı.

Aradan Silme — Karmaşıklık ve Özel Durumlar

prev ZATEN BİLİNİYORSA

`O(1) — sabit sayıda işlem → O(1)`

Önce ARAMA GEREKİYORSA

`O(n) — arama baskın gelir → O(n)`

- Boş liste (head==NULL): silinecek bir şey yok, HATA/erken dönüş gerekir.
- Tek düğümlü liste: silinen düğüm hem baş hem sondur — head, silme sonrası NULL olmalı.
- Baş düğüm siliniyorsa: “prev” yoktur — baştan silme algoritması (39-40. slayt) kullanılır.
- Son düğüm siliniyorsa: silme sonrası prev->sonraki mutlaka NULL yapılmalıdır.

Arama (Search)

Pseudocode

```
Fonksiyon Ara(head, deger)
    p ← head
    iken p ≠ NULL
        eğer p.veri == deger
            dön p
        p ← p.sonraki
    dön NULL   // bulunamadı
```

C Kodu

```
struct Dugum *ara(
    struct Dugum *head, int deger) {
    struct Dugum *p = head;
    while (p != NULL) {
        if (p->veri == deger) {
            return p;
        }
        p = p->sonraki;
    }
    return NULL;
}
```

Bağlı listede dizideki gibi “ortadan” başlayan (ikili) arama YAPILAMAZ — indeksli sıçrama imkanı olmadığı için sadece DOĞRUSAL arama mümkündür. Karmaşıklık: en kötü durumda O(n).

Karmaşıklık Özet Tablosu

İşlem	En İyi	En Kötü	Ön Koşul
Başa Ekleme	`O(1)`	`O(1)`	Yok
Sona Ekleme	`O(1)`	`O(n)`	O(1) sadece tail biliniyorsa
Araya Ekleme	`O(1)`	`O(n)`	O(1) sadece prev biliniyorsa
Baştan Silme	`O(1)`	`O(1)`	Yok
Sondan Silme	`O(n)`	`O(n)`	Tek yönlü listede her zaman
Aradan Silme	`O(1)`	`O(n)`	O(1) sadece prev biliniyorsa
Arama / Gezinme	`O(1)`	`O(n)`	Yok

Ortak tema: Ön koşul (“prev/tail zaten biliniyor mu?”) karmaşıklığı belirleyen en önemli faktördür.

Dizi vs Bağlı Liste — Güncellenmiş Karşılaştırma

2. haftadaki önizlemenin (slayt 43) tam versiyonu

Özellik	Dizi	Bağlı Liste
İndeksli erişim (dizi[i])	O(1)	O(n)
Başa ekleme/silme	O(n)	O(1)
Sona ekleme	O(1)	O(n) (tail yoksa)
Boyut	Oluşturulduktan sonra sabit	Dinamik olarak büyüyüp küçülebilir
Bellek fazlası	Yok	Her düğümde 1 işaretçi
Bellek yerleşimi	Bitişik	Dağınık

Doğru seçim, uygulamanın en sık yaptığı işleme bağlıdır: çok okuma/indeksleme → dizi; çok ekleme/silme → bağlı liste.

Çift Yönlü Bağlı Liste — Kavram

```
struct DugumCift {
    int veri;
    struct DugumCift *onceki;
    struct DugumCift *sonraki;
};
```

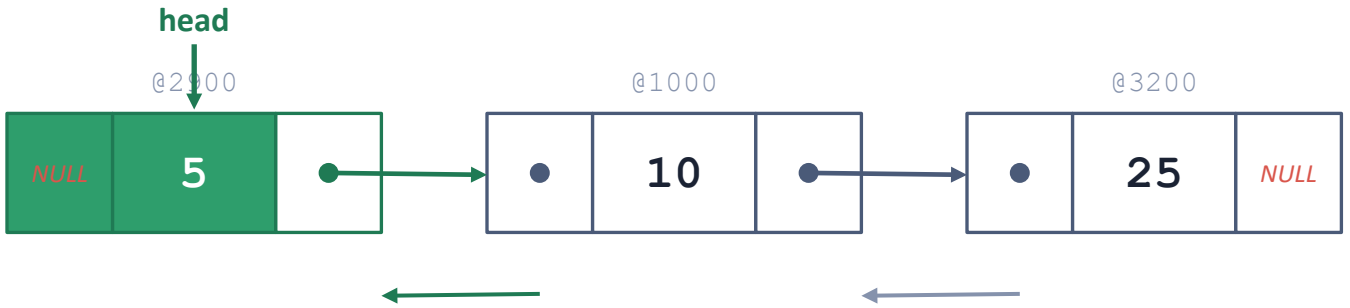
Her düğüm HEM önceki HEM sonraki düğümü bilir



Avantaj: GERİYE doğru da gezinilebilir. Bedel: her düğüm bir EK işaretçi (onceki) alanı gerektirir.

Çift Yönlü Liste — Başa Ekleme (Özet)

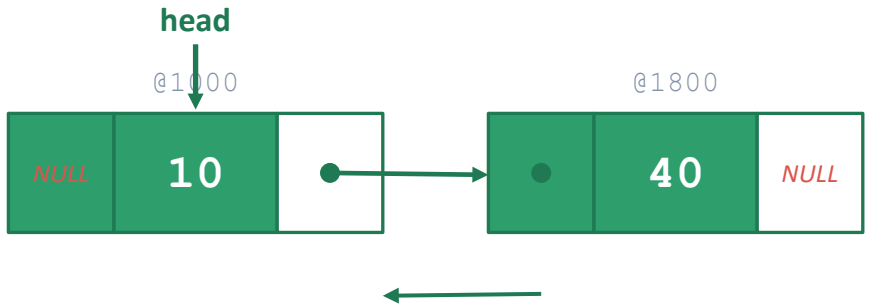
```
yeni->sonraki = head;
if (head != NULL) {
    head->onceki = yeni;    // ÇİFT YÖNLÜYE ÖZGÜ ekstra adım!
}
yeni->onceki = NULL;
head = yeni;
```



Tek yönlü listeden fark: yeni head'in onceki'sini NULL yapmanın yanı sıra, ESKİ head'in onceki'sini de yeni düğüme güncellemek gerekir — toplam 4 işaretçi işlemi.

Çift Yönlü Liste — Silme (Özet)

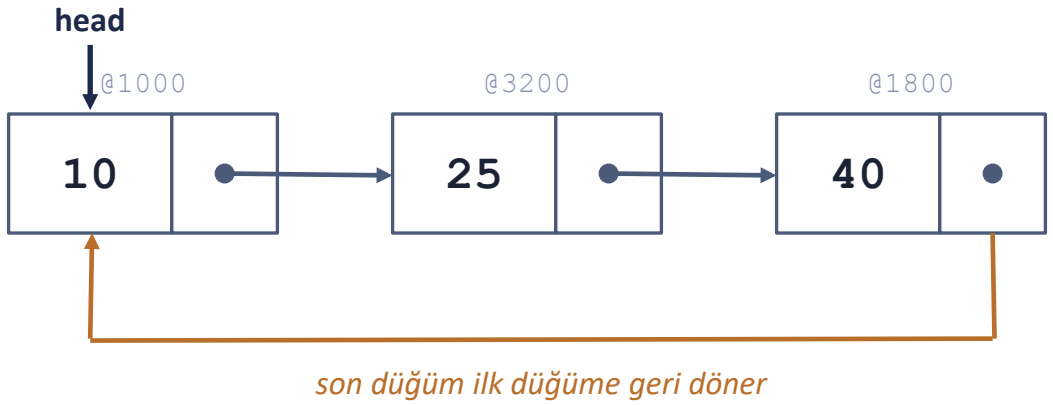
```
if (hedef->onceki != NULL)
    hedef->onceki->sonraki = hedef->sonraki;
else head = hedef->sonraki;          // hedef baştaydı
if (hedef->sonraki != NULL)
    hedef->sonraki->onceki = hedef->onceki;  // ÇİFT YÖNLÜYE ÖZGÜ
free(hedef);
```



Avantaj: Silme için önceki düğümü AYRICA takip etmeye gerek YOK — hedef->onceki zaten doğrudan erişilebilir.

Dairesel Bağlı Liste — Kavram

- Son düğüm NULL yerine TEKRAR İLK düğümü gösterir — liste bir “halka” oluşturur.



NOT: Bu diyagramda hala NULL yerine bir ok çizilmiştir; sadece görsel netlik için düz bir satırda gösterilmiştir. Gerçekte tüm düğümler aynı türden ve halka şeklinde bağlıdır — gerçek bir “baş” veya “son” kavramı yoktur, sadece bir giriş noktası (head) vardır.

Dairesel Liste — Gezinme Farkı

- **p ≠ NULL kontrolü ARTIK İŞE YARAMAZ — hiçbir düğüm NULL değil! Bunun yerine p ≠ head kontrolü kullanılır.**

Pseudocode

```
Fonksiyon DaireselGezin(head)
    eğer head == NULL
        dön
    p ← head
    yap
        yazdır p.veri
        p ← p.sonraki
    iken p ≠ head
```

C Kodu

```
void daireselGezin(
    struct Dugum *head) {
    if (head == NULL) return;
    struct Dugum *p = head;
    do {
        printf("%d->",p->veri);
        p = p->sonraki;
    } while (p != head);
}
```

Yaygın Hata: while(p!=NULL) ile dairesel listede gezinmeye çalışmak SONSUZ DÖNGÜYE yol açar — p asla NULL olmaz!

Bellek Sızıntısı (Memory Leak)

- **Bir düğüme giden TÜM işaretçiler kaybolduğunda ama düğüm free() edilmediyse, o bellek SONSUZA DEK kilitli kalır.**

Örnek: 37. slaytta gördüğümüz hatalı sıra, aynı zamanda bir bellek sızıntısı örneğidir

```
struct Dugum *p = yeniDugum(5);
p = yeniDugum(10);    // İLK düğümün adresi kaybedildi!
                      // O düğüm artık erişilemez ve free edilemez.
```

Korunma Yöntemleri: (1) Bir düğümü “atlamadan” önce daima adresini başka bir yerde saklayın. (2) Her malloc() için mutlaka karşılık gelen bir free() olsun. (3) Valgrind gibi araçlar bu tür sızıntıları çalışma zamanında tespit edebilir.

Sarkan İşaretçi (Dangling Pointer) ve NULL Kontrolü

- free() edilmiş bir belleği gösteren, ama HALA geçerliymiş gibi kullanılan işaretçiye “sarkan işaretçi” denir.

Tehlikeli örnek

```
free(p);
p->veri = 10;       // TANIMSIZ DAVRANIŞ! p artık geçersiz bir adresi gösteriyor
printf("%d", p->veri);  // Belki çalışır, belki çöker — GÜVENİLMEZ
```

free(p) sonrasında p = NULL yapılarak bu işaretçinin serbest bırakılmış belleği göstermesi önlenebilir. Ancak aynı belleği gösteren diğer işaretçiler varsa bunlar kendiliğinden NULL olmaz. NULL işaretçiyi kontrol etmeden kullanmak da tanımsız davranıştır.

İşaretçileri kullanmadan önce geçerliliklerini kontrol edin. Her dinamik bellek tahsisinin uygun zamanda serbest bırakıldığından emin olun. Serbest bırakılmış belleğe erişmeyin ve aynı belleği ikinci kez free() ile serbest bırakmayın.

İleri Düzey Ek Materyal

Kısaca tanıtım — detaylı uygulama bu haftanın kapsamı dışında

Çok Yönlü Listeler (Multi-linked)

Bir düğüm aynı anda BİRDEN FAZLA farklı sıralamada (ör. hem isme hem numaraya göre) zincirlenebilir — her sıralama için ayrı bir işaretçi alanı kullanılır.

Başlık Düğümlü Listeler (Header Node)

Gerçek veriyi tutmayan, sadece listenin başını işaret eden özel bir "nobanç" düğüm eklenir — boş liste/tek düğüm gibi özel durumları ortadan kaldırır.

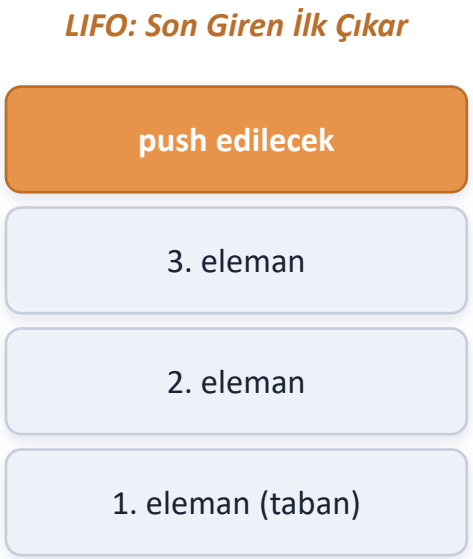
Bu iki yapı da, bu hafta gördüğümüz temel tek/çift yönlü liste kavramlarının doğrudan uzantılarıdır — aynı struct + işaretçi mantığı, sadece daha fazla işaretçi alanıyla. İleri veri yapıları derslerinde detaylı işlenir.

Haftalık Özet

- Bağlı listeyi ve malloc/free ile dinamik bellek yönetimini öğrendik.
- Tek yönlü listede başa/sona/araya ekleme ve silmeyi adım adım uyguladık.
- Yanlış işlem sırasının bağlantı kaybına (bellek sızıntısına) nasıl yol açtığını somut olarak gördük.
- Çift yönlü ve dairesel bağlı listelerin temel farklarını tanıdık.
- Bellek sızıntısı ve sarkan işaretçi hatalarından korunma yöntemlerini öğrendik.
- Tüm işlemlerin karmaşıklığını ön koşullarıyla (prev/tail biliniyor mu?) birlikte analiz ettik.

Gelecek Hafta Önizleme

4. Hafta: Yığınlar (Stacks)



- Yığın (stack) ADT: push, pop, top/peek işlemleri
- Dizi ile ve bağlı liste ile gerçekleştirim (iki farklı yaklaşım)
- **Bu hafta gördüğümüz başa ekleme/silme (O(1)), yığının bağlı liste ile doğal bir gerçekleştirimi olacak**
- Gerçek uygulamalar: fonksiyon çağrı yığını, geri alma (undo), parantez eşleştirme

Hazırlık: Bu haftaki "başa ekleme/silme O(1)" konusunu tekrar edin — 4. haftada bu doğrudan yığının temelini oluşturacak.

Kavrama Soruları / Mini Quiz

1) Bir düğümü araya eklerken neden önce yeni->sonraki, sonra prev->sonraki güncellenir? Ters sırada ne olur?

2) Baştan silme işleminde head'i ilerletmeden önce neden geçici bir değişkende (gecici) saklamak gerekir?

3) Dairesel bir listede while(p != NULL) yerine neden while(p != head) kullanılır?

4) Çift yönlü listede başa ekleme, tek yönlü listeye göre hangi EK adımı gerektirir?

Haftalık Ödev

- 1) struct Dugum tanımlayıp, kullanıcıdan alınan sayılarla bir tek yönlü bağlı liste oluşturan bir C programı yazınız (basaEkle/sonaEkle kullanarak).
- 2) [15, 22, 8, 40] değerleriyle oluşturulmuş bir listede, değer=22 olan düğümden SONRAYA 33 eklemeyi, bu haftaki slayt formatında (her adım ayrı) elle çözünüz.
- 3) Aynı listede değer=8 olan düğümü aradan silme işlemini elle, adım adım gösteriniz.
- **4) Bir tek yönlü bağlı listeyi TERSİNE çeviren (reverse) bir C fonksiyonu yazınız. (İpucu: 3 işaretçi — onceki, simdi, sonraki — kullanın.)**

Teslim: Bir sonraki ders öncesi, C kaynak dosyaları + elle çözülmüş adımları gösteren bir PDF olarak.

Kaynakça

Kaynak	İlgili Bölüm(ler)	Sayfa Aralığı
Horowitz, Sahni, Anderson-Freed — Fundamentals of Data Structures in C	Chapter 4: Lists	s. 135–185
Thareja — Data Structures Using C	Chapter 6: Linked Lists	s. 162–218

Not:

Ekleme/silme adım adım diyagramları, hata durumu (bağlantı kaybı) gösterimi, dairesel liste gezinme örneği ve bellek güvenliği örnekleri özgün olarak hazırlanmıştır; kaynaklardaki ilgili bölümler kavramsal çerçeve için referans alınmıştır.